

9/15/26 Intro to Alignment Algorithms: Global and Local

- Global Alignment (easiest)
- Local Alignment (most-used algorithm in all of computational biology)

Sequence Comparison

- Biomolecular sequences
 - DNA sequences (A, T, G, C bases)
 - RNA sequences (A, U, G, C bases)
 - Protein sequences (20 amino acids)
 - Sequence similarity helps in the discovery of genes

Global Similarity

- Scoring schemes
- Edit graphs
- Alignment = path in the edit graph
- Principle of optimality
- Dynamic programming algorithm
- The Traceback

Sequence Alignment

- Input: two sequences over the same alphabet
- Output: alignment of the two sequences
- Ex:
 - GCGCATTTGAGCGA
 - TGCTTAGGGTGACCA
 - Possible alignment
 - -GCGCATTTGAGCGA - -
 - TGCG- -TTAGGGTGACC
 - Match = letters match perfectly
 - Mismatch = different letter
 - Indel = lack of matching letters present due to insertion or deletion
- Unit-scoring scheme
 - In alignment, align letter with another letter represented by a matrix

	A	T	C	G
A	1	0	0	0
T	0	1	0	0
C	0	0	1	0
G	0	0	0	1

- The above is an alignment of DNA sequences, but if for proteins, would be an alignment of 20 different AAs
- Alignment
 - Score is the sum of pairwise letter assignments
 - E.g., delta of A and A + delta of C and G
 - If we add gaps, there is a higher score because more is revealed
 - Scoring scheme
 - $\delta(x,y)$
 - Score: $\delta_{(A,A)} + \delta_{(C,T)}$
- Dayhoff score = sophisticated scoring scheme for protein sequences
- Scoring Functions
 - Mutations = substitutions, insertions, deletions

The Edit Graph

- How to find absolute best alignment
- Where alignments between two sequences correspond to paths between the beginning and end nodes of the graph
- Exponential
- Eg: aligning AGT with AT
 - AGT has length 3, AT has length 2
 - Start with a grid with 2 rows for AT and 3 col for AGT
 - Then label
 - The edit graph has $(3+1)*(2+1)$ nodes

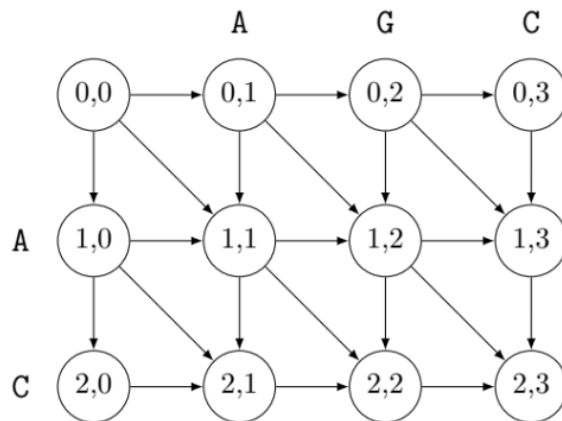


Figure 2. Edit Graph for AC and AGC.

- The graph is directed. The nodes (i,j) will hold values
- Directed edges give us alignment; they are labels pairs of aligned letters
- Vertical edges go from top to bottom and bc there's no advancement in the other sequence so can put a gap
 - Same goes for if we're going horizontally so we can put a gap for the lack of advancement in the other sequence
 - Every path from beginning to end represents an alignment
 - Every alignment corresponds to a path between beginning to end

The Principle of Optimality

- The optimal answer to a problem is expressed in terms of optimal answer for its subproblems
- The optimal path can be made of suboptimal paths

Dynamic Programming

- Given: two sequences X and Y and a scoring scheme δ
- Find: an optimal alignment of X and Y (max score alignment)
 - Part 1: compute first the optimal alignment score
 - Part 2: construct optimal alignment
 - Called a traceback b/c we'll go back from right to left and actually construct the path one pair of letters at a time
 - We're looking for the optimal alignment = maximal score path in the edit graph from the beginning vertex to the end vertex
- DP Matrix $S(i,j)$
 - Vertical, diagonal, horizontal = those are the 3 that inspire the algorithm
 - Matrix $S = [S(i,j)]$
 - $S(i,j)$ = the score of the maximal cost path from the Begin vertex and the vertex (i,j)
 - The optimal path to (i,j) must pass through one of the vertices
 - $(i-1,j)$
 - $(i,j-1)$
 - $(i-1,j-1)$
 - Opt path to $(i-1,j-1) + \delta(x_i, y_j)$
 - Opt path to $(i-1,j) + \delta(-, y_j)$
 - Opt path to $(i,j-1) + \delta(x_i, -)$
- The Basic Algorithm
 - $S(i,j) = \text{MAX of } \{$
 - $S(i-1,j-1) + \delta(s_i, y_j),$
 - $S(i-1,j) + \delta(x_i, -),$
 - $S(i,j-1) + \delta(-, y_j) \}$
 - Quadratic algorithm
 - Need to fill $n*m$ matrix to get the optimal solution
 - Evolution is captured in scoring schemes

Local Similarity (Basic algorithm)

- $S(i,j) = \text{MAX of } \{$
 - $0,$
 - $S(i-1,j-1) + \delta(s_i, y_j),$
 - $S(i-1,j) + \delta(x_i, -),$
 - $S(i,j-1) + \delta(-, y_j) \}$

General Scoring Schemes

- Assumptions
 1. Independence of mutations at different sites
 - a. Additive scoring scheme

- 2. Gaps of any length are considered one mutation
- All of the efficient alignment algorithms - employing the dynamic programming method - are based fundamentally on the fact that the scoring function is additive
- Entries in scoring schemes are statistical tests of significance
 - Heuristic interpretation of the alignment score
 - $X = \text{AACGTAA}$
 - Y (sequence we want to align) = ACGGTGT

A A C G - T A A

| | | |

A - C G G T G T

4 matches, 2 gaps, 2 mismatches

Let p be the probability of a match

Let q be the probability of a substitution/mismatch

Let r be the probability of a single-letter indel gap

What is the probability of the alignment(*)?

$$P_{\text{alignment}(*)} = p^4 q^2 r^2$$

The log-likelihood is

$$s' = \log P_{\text{alignment}(*)} = 4 \log p + 2 \log q + 2 \log r$$

$$s = s' - 8 \log k \text{ and } k = \text{constant}$$

$$s = s' - 8 \log k = 4 \log p + 2 \log q + 2 \log r - 8 \log k = 4 \log (p/k) + 2 \log (q/k) + 2 \log (r/k) = 4 - 2\mu - 2\gamma$$

$$\mu = \text{penalty of mismatch} = \log(k/q)$$

$$\gamma = \text{penalty for gap} = \log(k/r)$$

$$\text{Chose } k = \log(p/k) = 1$$